

# Rapport Cas Pratique: Sélection du Service OCR

Cas Pratique - Certification RNCP37827BC02

Jean Luis Soto

2026-02-19

## Table of contents

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>Introduction et Expression du Besoin</b>      | <b>2</b> |
| 1.1      | Contexte du Projet . . . . .                     | 2        |
| 1.2      | Expression du Besoin IA . . . . .                | 2        |
| <b>2</b> | <b>Dispositif de Veille Technologique</b>        | <b>2</b> |
| <b>3</b> | <b>Inventaire et Préconisation</b>               | <b>3</b> |
| 3.1      | Inventaire des Solutions Accessibles . . . . .   | 3        |
| 3.2      | Préconisation Technique . . . . .                | 3        |
| <b>4</b> | <b>Résultats du Benchmark et Sélection Final</b> | <b>3</b> |
| <b>5</b> | <b>Paramétrage et Optimisation (HPO)</b>         | <b>4</b> |
| 5.1      | Optimisation Bayésienne avec Optuna . . . . .    | 4        |
| <b>6</b> | <b>Guide d'Installation et Configuration</b>     | <b>5</b> |
| 6.1      | Prérequis . . . . .                              | 6        |
| 6.2      | Étapes de Configuration . . . . .                | 6        |
| <b>7</b> | <b>Suite des Travaux</b>                         | <b>6</b> |

# 1 Introduction et Expression du Besoin

## 1.1 Contexte du Projet

Le commanditaire est une entreprise SaaS ERP **SINTAD**, spécialisée dans le secteur douanier au Pérou. Les utilisateurs doivent saisir de nombreuses données techniques à partir de factures d'importation complexes. Le processus manuel actuel est jugé **fastidieux, lent et sujet aux erreurs humaines**.

## 1.2 Expression du Besoin IA

Le besoin identifié résulte d'une sollicitation interne visant à automatiser l'extraction de données structurées. Une tentative précédente avec Google Document AI a révélé des limites en termes de coûts et de flexibilité (sur-apprentissage).

Les objectifs critiques sont :

1. **Automatisation Robuste** : Sans ré-entraînement constant par client.
2. **Haute Précision** : Fiabilité élevée pour les champs critiques (montants, dates, articles).
3. **Performance (SLA)** : Traitement de **50 factures en moins de 5 minutes**.
4. **Optimisation des Coûts** : Réduction des coûts opérationnels par rapport aux solutions propriétaires.

## 2 Dispositif de Veille Technologique

Le dispositif de veille a reposé sur une investigation approfondie des solutions existantes à travers des revues technologiques telles que **Medium** (Towards Data Science) et des dépôts de référence comme **Kaggle**.

Une revue systématique des modèles open-source sur **Hugging Face** a été effectuée, complétée par l'analyse des recommandations techniques de la **Linux Foundation**. Cette démarche a permis d'identifier :

- Des solutions open-source matures : **Tesseract**, **EasyOCR**.
- Des technologies innovantes : la bibliothèque **Docling** d'IBM.
- Des modèles de pointe (State-of-the-Art) : **Llama Maverick**, **DeepSeek OCR**.
- Des solutions Cloud : **Google Cloud Vision**, **Mistral AI**, **OpenAI**.

Les tests préliminaires ont révélé que si les modèles open-source sont performants, ils exigent souvent une puissance de calcul importante (GPU) pour garantir la rapidité de traitement ou présentent une précision insuffisante sur des documents complexes sans une configuration lourde.

### 3 Inventaire et Préconisation

#### 3.1 Inventaire des Solutions Accessibles

Un inventaire des services répondant à la problématique a été réalisé, classé par type d'accès :

| Type                  | Services Identifiés                 | caractéristiques                                            |
|-----------------------|-------------------------------------|-------------------------------------------------------------|
| Logiciels Libres (OS) | Tesseract, EasyOCR, Docling         | Gratuit, exécution locale, nécessite post-traitement Regex. |
| Services Cloud (SaaS) | GCP Vision, AWS Textract            | Précis, coûteux à l'usage, "Boîte noire".                   |
| VLM (Vision LLM)      | Mistral OCR, Llama 3 (Groq), GPT-4o | Compréhension sémantique, sortie JSON native.               |

#### 3.2 Préconisation Technique

À la suite de l'inventaire, il a été préconisé d'utiliser une solution basée sur **Mistral AI** ou **Llama 3 (via Groq)**. Ces services offrent le meilleur compromis entre vitesse (via LPU), précision contextuelle et coût. **Mistral** a été retenu comme solution principale pour sa robustesse multilingue, tandis que **Tesseract** est conservé comme "Baseline" de comparaison.

### 4 Résultats du Benchmark et Sélection Final

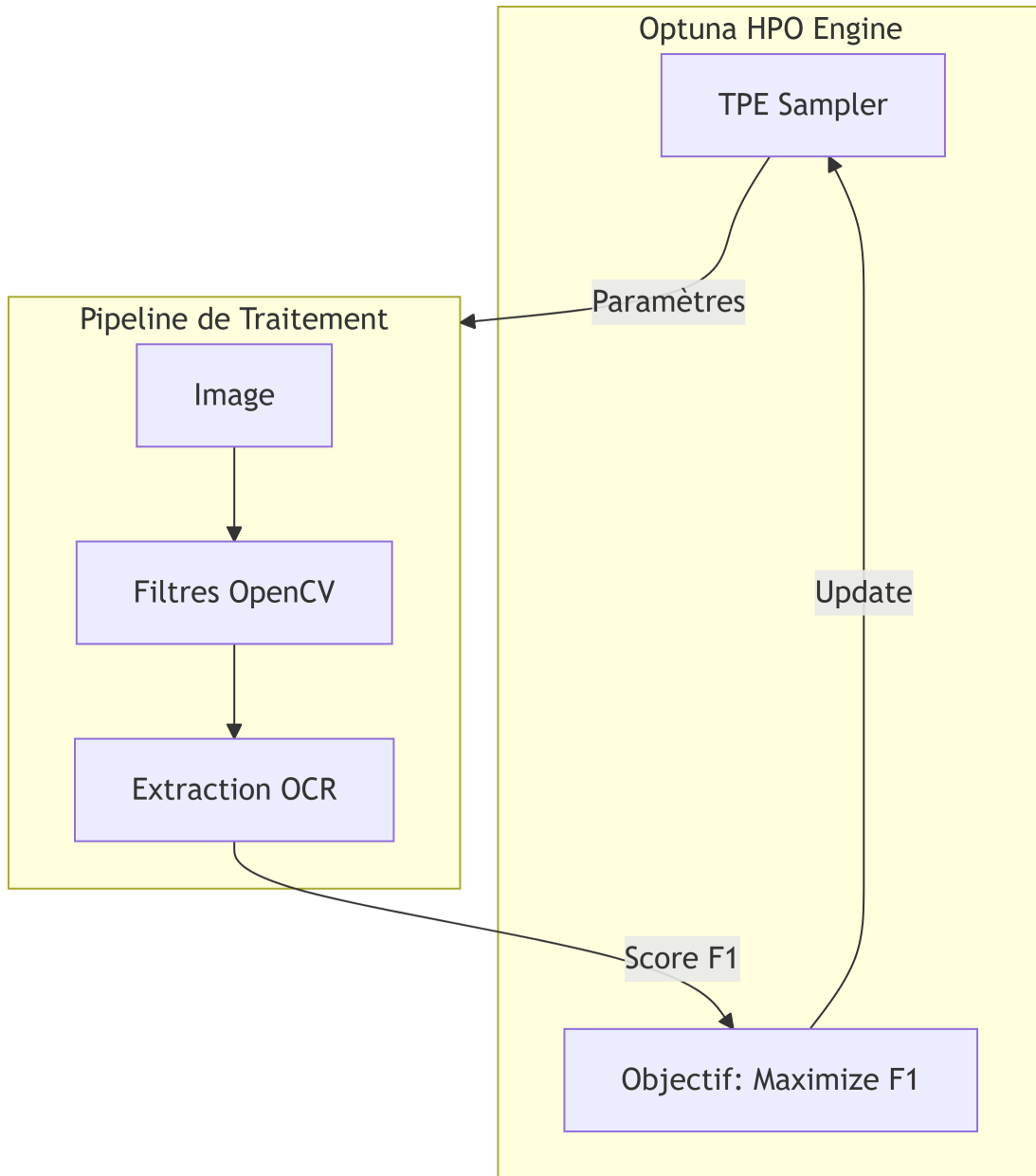
| Modèle                | Précision (F1) | Vitesse             | Conclusion                                          |
|-----------------------|----------------|---------------------|-----------------------------------------------------|
| <b>Tesseract 5</b>    | ~60%           | Rapide              | Insuffisant pour les tableaux complexes.            |
| <b>Llama 3 (Groq)</b> | 90%            | <b>Ultra-Rapide</b> | Excellente alternative pour le SLA.                 |
| <b>Mistral Large</b>  | <b>95%</b>     | Moyenne             | <b>Solution retenue</b> pour la mise en production. |

## 5 Paramétrage et Optimisation (HPO)

Pour garantir la qualité de l'OCR, une couche de paramétrage intelligent a été configurée.

### 5.1 Optimisation Bayésienne avec Optuna

Plutôt que de définir les paramètres de nettoyage d'image de manière empirique, une **optimisation bayésienne des hyperparamètres (HPO)** a été implémentée avec **Optuna**. Ce service ajuste dynamiquement le contraste, le débruitage et la rotation pour maximiser le F1-Score global.



## 6 Guide d'Installation et Configuration

Pour assurer la mise en service technique demandée, les étapes suivantes sont nécessaires.

## 6.1 Prérequis

- **Docker** et **Docker Compose** installés.
- Accès Internet pour les API (Mistral/Groq).

## 6.2 Étapes de Configuration

1. **Clonage du dépôt :**

```
git clone [URL_DU_REPO]
cd ocr_comparison
```

2. **Configuration de l'environnement :** Créer un fichier `.env` à la racine :

```
MISTRAL_API_KEY=votre_cle_ici
GROQ_API_KEY=votre_cle_ici
MLFLOW_TRACKING_URI=http://localhost:5000
```

3. **Installation via Docker :**

```
docker-compose up --build
```

Cette commande installe automatiquement les dépendances Python, configure l'API FastAPI et lance le serveur de tracking MLflow.

4. **Vérification :** Accéder à `http://localhost:8000/docs` pour tester les endpoints de l'API.

## 7 Suite des Travaux

Pour découvrir comment ce pipeline a été industrialisé et déployé sur GCP, consultez la deuxième partie du rapport :

[Partie 2/2 : Industrialisation et Déploiement](#)